

Full-Stack Developer

Interview Preparation Guide

MERN Stack & Next.js | Mid-Level & Above

7 Phases | 100+ Questions | Basic to Advanced

Curated by N.I. Nazmul

Full-Stack Developer (MERN Stack & Next.js Specialist)

github.com/ninazmul

Phase 1 JavaScript & TypeScript	Phase 2 React & Next.js	Phase 3 Node.js & Express
Phase 4 MongoDB & PostgreSQL	Phase 5 System Design & DSA	Phase 6 Architecture & DevOps
Phase 7 Testing & QA		

Phase 1: JavaScript & TypeScript Fundamentals

21 Q&As

Q1

BASIC

What is the difference between var, let, and const?

Answer:

var is function-scoped and hoisted (initialized to undefined). let and const are block-scoped and live in the Temporal Dead Zone until declared. const cannot be reassigned, but its object/array contents can still be mutated.

```
var x = 1;
if (true) {
  var x = 10; // same variable – function-scoped
  let y = 20; // block-scoped, dies here
}
console.log(x); // 10
// console.log(y); // ReferenceError
```

Interview tip: Prefer const by default; use let when reassignment is needed; avoid var in modern code.

Q2

BASIC

What is hoisting in JavaScript?

Answer:

Hoisting moves variable and function declarations to the top of their scope before execution. var is hoisted and initialized to undefined. Function declarations are fully hoisted. let and const are hoisted but stay in the Temporal Dead Zone (TDZ) — accessing them before declaration throws a ReferenceError.

```
console.log(a); // undefined (var hoisted)
var a = 5;

greet(); // works – function fully hoisted
function greet() { return 'hello'; }

console.log(b); // ReferenceError (TDZ)
let b = 10;
```

Interview tip: The TDZ is the gap between entering scope and the actual declaration line.

Q3

BASIC

Explain JavaScript closures with an example.

Answer:

A closure is a function that retains access to its outer scope even after that outer function has finished executing. Closures enable data encapsulation, factory functions, and stateful logic.

```
function makeCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count,
  };
}
const c = makeCounter();
c.increment(); c.increment();
```

```
console.log(c.value()); // 2
```

Interview tip: Each call to makeCounter() creates an independent closure with its own count.

Q4

BASIC

What is the difference between == and ===?

Answer:

== (loose equality) coerces types before comparing. === (strict equality) compares value AND type with no coercion. Always use === in production code to avoid subtle type-coercion bugs.

```
0 == false      // true  (coercion)
0 === false     // false (different types)
null == undefined // true
null === undefined // false
'' == 0         // true
'' === 0        // false
```

Interview tip: Use === everywhere. The only valid use of == is null-check: x == null catches both null and undefined.

Q5

BASIC

What are JavaScript primitive types?

Answer:

Primitives: string, number, bigint, boolean, undefined, null, symbol. Reference type: object (arrays, functions, maps, sets). Primitives are copied by value; objects are copied by reference.

```
typeof 'hi'      // 'string'
typeof 42        // 'number'
typeof true      // 'boolean'
typeof undefined // 'undefined'
typeof null      // 'object' (JS bug)
typeof []        // 'object'
typeof function(){} // 'function'
```

Interview tip: typeof null === 'object' is a historical JavaScript bug — use null === x to check for null.

Q6

BASIC

What is the difference between null and undefined?

Answer:

undefined means a variable has been declared but not assigned a value — it's the default empty value. null is an intentional assignment meaning 'no value'. typeof undefined is 'undefined'; typeof null is 'object' (historical bug).

```
let a;           // undefined automatically
let b = null;    // intentional empty

function getUser(id) {
  if (!id) return null; // intentional: no user
}

console.log(a == b); // true (loose)
console.log(a === b); // false (strict)
```

Interview tip: Use null when you intentionally want to represent 'no value'. Let undefined arise naturally.

Explain the JavaScript event loop.**Answer:**

JavaScript is single-threaded. The event loop manages async operations via the call stack, Web APIs, microtask queue, and callback (macro-task) queue. Microtasks (Promises, queueMicrotask) always run before macrotasks (setTimeout, setInterval) — the microtask queue is fully drained after each task.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
// Microtask (3) runs before macrotask (2)
```

Interview tip: setTimeout(fn, 0) does not mean immediate — it means 'run after current + microtasks finish'.

What is the difference between Promise, async/await, and callbacks?**Answer:**

Callbacks lead to callback hell when nested. Promises provide cleaner chaining with .then().catch(). async/await is syntactic sugar over Promises — makes async code read synchronously, easier to debug and reason about.

```
// Callback hell
fetchUser(id, (err, user) => {
  fetchPosts(user.id, (err, posts) => {
    fetchComments(posts[0].id, (err, comments) => {});
  });
});

// async/await (clean)
async function load(id) {
  try {
    const user = await fetchUser(id);
    const posts = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);
  } catch(e) { console.error(e); }
}
```

Interview tip: Always handle rejections. Unhandled promise rejections crash Node.js processes.

Explain prototypal inheritance in JavaScript.**Answer:**

Every JS object has an internal [[Prototype]] link. When accessing a property, JS walks the prototype chain until it finds it or reaches null. ES6 classes are syntactic sugar over this mechanism.

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() {
  return this.name + ' speaks';
};

const dog = new Animal('Rex');
console.log(dog.speak()); // Rex speaks
console.log(dog.__proto__ === Animal.prototype); // true
console.log(dog.hasOwnProperty('speak')); // false
```

Q10

INTERMEDIATE

Explain the classic var + closure loop problem.

Answer:

With `var` in a `for` loop, all callbacks share the same variable reference. By execution time, `i` holds its final value. Fix: use `let` (block-scoped per iteration) or an IIFE to capture each value.

```
// Problem – prints 3,3,3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

// Fix 1: let (block-scoped)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 0,1,2
}

// Fix 2: IIFE
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j), 0))(i);
}
```

Interview tip: This is one of the most common closure interview questions — know both fixes.

Q11

INTERMEDIATE

What is the difference between `call`, `apply`, and `bind`?

Answer:

All three set the `this` context. `call()` invokes immediately with args as a list. `apply()` invokes immediately with args as array. `bind()` returns a new function with `this` permanently bound — useful for event handlers and partial application.

```
function greet(greeting, punct) {
  return greeting + ', ' + this.name + punct;
}
const user = { name: 'Nazmul' };

greet.call(user, 'Hello', '!'); // 'Hello, Nazmul!'
greet.apply(user, ['Hi', '?']); // 'Hi, Nazmul?'
const fn = greet.bind(user, 'Hey');
fn('.'); // 'Hey, Nazmul.'
```

Interview tip: `bind` is commonly used to preserve `this` in class methods passed as React event handlers.

Q12

INTERMEDIATE

What are `Promise.all`, `Promise.race`, `Promise.allSettled`, and `Promise.any`?

Answer:

`Promise.all`: resolves when all resolve; rejects if any reject. `Promise.race`: resolves/rejects as soon as first settles. `Promise.allSettled`: waits for all — never rejects, returns status of each. `Promise.any`: resolves with first success; rejects only if all fail.

```
const p1 = fetch('/api/users');
const p2 = fetch('/api/posts');

// All must succeed
const [users, posts] = await Promise.all([p1, p2]);
```

```
// Results regardless of failure
const results = await Promise.allSettled([p1, p2]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.log(r.reason);
});
```

Interview tip: Use Promise.allSettled when you need results from all — even failed — promises.

Q13

INTERMEDIATE

What are arrow functions and how do they differ from regular functions?

Answer:

Arrow functions are concise, don't have their own this (inherit from enclosing scope), no arguments object, cannot be used as constructors, and don't have a prototype property. Perfect for callbacks and array methods.

```
// Regular function – has own 'this'
function Timer() {
  this.count = 0;
  setInterval(function() {
    this.count++; // 'this' is undefined in strict
  }, 1000);
}

// Arrow function – inherits 'this'
function Timer() {
  this.count = 0;
  setInterval(() => {
    this.count++; // 'this' is Timer instance
  }, 1000);
}
```

Interview tip: Never use arrow functions as object methods if you need this to refer to the object.

Q14

ADVANCED

What are WeakMap and WeakSet? When would you use them?

Answer:

WeakMap and WeakSet hold weak references — they don't prevent garbage collection of keys/values. Keys must be objects. Neither is iterable. Used for: attaching private metadata to objects, caching without memory leaks, tracking presence without preventing GC.

```
const cache = new WeakMap();

function process(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const result = heavyComputation(obj);
  cache.set(obj, result); // auto-removed when obj is GC'd
  return result;
}

// Regular Map would prevent GC of obj
// WeakMap allows GC – no memory leak
```

Interview tip: WeakMap is the correct tool for associating data with DOM nodes without memory leaks.

Q15**ADVANCED****Explain TypeScript generics.****Answer:**

Generics write reusable, type-safe code that works with multiple types. Instead of any (loses type info), generics capture the type at call time and propagate it through the function or class.

```
function identity<T>(arg: T): T { return arg; }

// Real-world: typed API response
async function fetchData<T>(url: string): Promise<T> {
  const res = await fetch(url);
  return res.json() as T;
}

interface User { id: number; name: string; }
const user = await fetchData<User>('/api/user/1');
user.name; // TypeScript knows this is string
```

Interview tip: Use constraints (T extends object) to limit which types can be passed.

Q16**ADVANCED****What are TypeScript utility types?****Answer:**

Utility types transform existing types. Partial makes all optional. Required makes all mandatory. Pick selects keys. Omit removes keys. Record maps key to value type. Readonly prevents mutation.

```
interface User {
  id: number; name: string;
  email: string; age: number;
}

type UpdateUser = Partial<User>;
// All fields optional – for PATCH requests

type UserPreview = Pick<User, 'id' | 'name'>;
// Only id and name

type PublicUser = Omit<User, 'email'>;
// Everything except email

type UserMap = Record<string, User>;
// { [key: string]: User }
```

Interview tip: Combine utility types: Partial for partial update DTOs.

Q17**ADVANCED****What is the difference between interface and type in TypeScript?****Answer:**

Both define shapes. Interfaces support extends and declaration merging (same interface declared twice merges). Types are more flexible — can represent unions, intersections, tuples, and primitives. Interfaces for object shapes; types for unions and complex compositions.

```
interface Animal { name: string; }
interface Dog extends Animal { breed: string; }

// Declaration merging (interfaces only)
interface Window { myProp: string; }
interface Window { another: number; } // merged
```

```
// Types handle unions
type ID = string | number;
type Result<T> = { data: T } | { error: string };
type Point = [number, number]; // tuple
```

Interview tip: In a codebase, pick one and be consistent. Most teams use interface for objects, type for the rest.

Q18

ADVANCED

Explain JavaScript memory management and memory leak patterns.

Answer:

JS uses mark-and-sweep garbage collection. Memory leaks occur when references are unintentionally kept alive. Common causes: forgotten event listeners, closures holding large objects, global variables, detached DOM nodes in variables, and uncleaned setInterval.

```
// LEAK: listener never removed
function setup() {
  const bigData = new Array(1e6).fill('x');
  document.addEventListener('click', () => {
    console.log(bigData.length); // bigData never GC'd
  });
}

// FIX: remove listener
const handler = () => console.log('clicked');
document.addEventListener('click', handler);
// Cleanup:
document.removeEventListener('click', handler);
```

Interview tip: Use Chrome DevTools Memory tab and heap snapshots to identify leaks in production.

Q19

ADVANCED

What is the JavaScript Proxy object?

Answer:

Proxy wraps an object and intercepts fundamental operations (get, set, delete, apply). Reflect provides the same operations as static methods. Together they enable reactive systems, validation, logging — Vue 3's reactivity is built on Proxy.

```
const handler = {
  get(target, key) {
    console.log('Getting: ' + key);
    return Reflect.get(target, key);
  },
  set(target, key, value) {
    if (typeof value !== 'number')
      throw new TypeError('Must be number');
    return Reflect.set(target, key, value);
  }
};
const state = new Proxy({}, handler);
state.count = 5; // validated
state.count;    // logs 'Getting: count'
```

Interview tip: Proxy is how Vue 3 implements reactivity — understanding it is a strong senior signal.

What is the difference between shallow copy and deep copy?

Answer:

Shallow copy duplicates the top level but nested objects share references. Deep copy recursively duplicates everything — fully independent. Object spread and Object.assign are shallow. structuredClone() is the modern native deep copy.

```
const orig = { a: 1, b: { c: 2 } };

// Shallow – b is shared
const shallow = { ...orig };
shallow.b.c = 99;
console.log(orig.b.c); // 99 (mutated!)

// Deep – fully independent
const deep = structuredClone(orig);
deep.b.c = 99;
console.log(orig.b.c); // 2 (safe)
```

Interview tip: JSON.parse(JSON.stringify()) fails on Date, undefined, functions, and circular refs — use structuredClone.

Explain JavaScript generators.

Answer:

Generator functions (function*) produce iterators via yield. They enable lazy evaluation, infinite sequences, and custom iteration. When you call next(), execution runs until the next yield and pauses.

```
function* range(start, end, step = 1) {
  for (let i = start; i < end; i += step) yield i;
}

for (const n of range(0, 10, 2)) {
  console.log(n); // 0, 2, 4, 6, 8
}

function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) { yield a; [a,b] = [b, a+b]; }
}

const fib = fibonacci();
fib.next().value; // 0
fib.next().value; // 1
```

Interview tip: Generators power async iteration (for await...of) and are the foundation of Redux-Saga.

Q22

BASIC

What is the Virtual DOM?

Answer:

The Virtual DOM is a lightweight JS representation of the real DOM. React creates a new virtual tree on state change, diffs it with the previous one (reconciliation), and updates only changed nodes in the real DOM — minimizing expensive DOM operations.

```
// React handles diffing automatically:
function Counter() {
  const [n, setN] = useState(0);
  return (
    <button onClick={() => setN(c => c+1)}>
      Count: {n}
    </button>
  );
  // Only the text node updates, not the whole button
}
```

Interview tip: React's diffing algorithm assumes same-type elements at the same position are the same component.

Q23

BASIC

What is the difference between controlled and uncontrolled components?

Answer:

Controlled: form data is driven by React state — every change fires onChange and updates state. Uncontrolled: data lives in the DOM, accessed via refs. Controlled is preferred for validation and conditional logic.

```
// Controlled
function Controlled() {
  const [val, setVal] = useState('');
  return (
    <input
      value={val}
      onChange={e => setVal(e.target.value)}
    />
  );
}

// Uncontrolled
function Uncontrolled() {
  const ref = useRef();
  const submit = () => console.log(ref.current.value);
  return <input ref={ref} />;
}
```

Interview tip: File inputs are always uncontrolled — their value is read-only in the DOM.

What is the difference between `useEffect` and `useLayoutEffect`?

Answer:

`useEffect` runs asynchronously after the browser paints. `useLayoutEffect` runs synchronously after DOM mutations but before paint. Use `useLayoutEffect` when you need to read/mutate layout (measure size, avoid visual flicker). Prefer `useEffect` by default.

```
// useEffect – after paint (async, no flicker risk)
useEffect(() => {
  document.title = 'Updated: ' + count;
}, [count]);

// useLayoutEffect – before paint (sync)
useLayoutEffect(() => {
  const { height } = ref.current.getBoundingClientRect();
  setHeight(height); // set before user sees anything
}, []);
```

Interview tip: Only reach for `useLayoutEffect` when you see visual flickering with `useEffect`.

Explain `useMemo` and `useCallback` — when to use them.

Answer:

`useMemo` memoizes a computed value. `useCallback` memoizes a function reference. Both only re-compute/re-create when dependencies change. Use them to prevent child re-renders or avoid expensive recalculations. Do not use prematurely — measure first.

```
// useMemo – skip re-sorting on unrelated renders
const sorted = useMemo(
  () => [...items].sort((a,b) => a.price - b.price),
  [items]
);

// useCallback – stable ref for memo'd child
const handleClick = useCallback(
  () => dispatch({ type: 'INCREMENT' }),
  [dispatch]
);

<ExpensiveChild onClick={handleClick} />
```

Interview tip: `useMemo/useCallback` have overhead too. Add them only after profiling shows a real problem.

How does React Context API work? What are its limitations?

Answer:

Context passes data through the tree without prop drilling. Create context, wrap with `Provider`, consume with `useContext`. Limitation: any `Provider` value change re-renders ALL consumers — even those not using the changed value. For high-frequency state, use `Zustand` or `Redux` instead.

```
const ThemeCtx = createContext('light');

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeCtx.Provider value={{ theme, setTheme }}>
```

```

    <Layout />
  </ThemeCtx.Provider>
);
}

function Button() {
  const { theme } = useContext(ThemeCtx);
  return <button className={theme}>Click</button>;
}

```

Interview tip: Split context by concern — separate ThemeContext from UserContext to avoid unnecessary re-renders.

Q27

INTERMEDIATE

What role do keys play in React lists?

Answer:

Keys help React identify which items changed, were added, or removed during reconciliation. Bad keys (array index) cause bugs when items reorder or have local state. Always use stable, unique IDs.

```

// Bad – using index as key
items.map((item, i) => <Item key={i} data={item} />);
// If items reorder, React thinks content
// changed in place – local state is wrong

// Good – stable unique ID
items.map(item => (
  <Item key={item.id} data={item} />
));

```

Interview tip: Keys must be stable across renders, unique among siblings, and not randomly generated at render time.

Q28

INTERMEDIATE

Explain Next.js rendering: SSR, SSG, ISR, CSR.

Answer:

SSR: HTML generated per request — real-time, personalized. SSG: HTML built at compile time — fastest, great for blogs/docs. ISR: SSG with timed revalidation — content stays fresh without full rebuilds. CSR: rendered in browser — used for dashboards behind auth.

```

// SSG (force-cache = static)
async function Page() {
  const data = await fetch(url, { cache: 'force-cache' });
  return <Render data={data} />;
}

// ISR – revalidate every 60 seconds
async function Page() {
  const data = await fetch(url, { next: { revalidate: 60 } });
  return <Render data={data} />;
}

// SSR – always fresh
async function Page() {
  const data = await fetch(url, { cache: 'no-store' });
  return <Render data={data} />;
}

```

Interview tip: ISR is the default recommendation for most marketing and content pages.

What is the difference between Server Components and Client Components?

Answer:

Server Components (default in App Router) render on the server, can access databases directly, have no client JS cost, but cannot use hooks or browser APIs. Client Components ('use client') run in the browser, support hooks and events, but add to the JS bundle.

```
'use client';
import { useState } from 'react';

// Client Component
export function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n+1)}>{n}</button>;
}

// Server Component (no directive)
async function UserList() {
  const users = await db.user.findMany();
  return users.map(u => <li key={u.id}>{u.name}</li>);
}
```

Interview tip: Server Components reduce JS bundle size. Push 'use client' as far down the tree as possible.

How do you prevent unnecessary re-renders?

Answer:

1) React.memo — skip re-render if props haven't changed. 2) useCallback — stable function refs. 3) useMemo — memoize derived values. 4) State colocation — keep state close to where it's used. 5) Context splitting — separate fast-changing from slow-changing context. 6) Virtualize long lists.

```
// React.memo
const Card = React.memo(({ user }) => (
  <div>{user.name}</div>
));

// Colocate state – don't lift unnecessarily
function Form() {
  // This state only affects Form
  const [value, setValue] = useState('');
  return <input value={value} onChange={e => setValue(e.target.value)} />;
}
```

Interview tip: Use React DevTools Profiler to identify what's actually re-rendering before optimizing.

Explain useReducer and when to prefer it over useState.

Answer:

useReducer manages complex state via a pure reducer function. Prefer it when: state has multiple interrelated sub-values, transitions are complex, or you want testable state logic separate from components.

```
type Action =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'RESET' };
```

```
function reducer(state: number, action: Action) {
  switch (action.type) {
    case 'INCREMENT': return state + 1;
    case 'DECREMENT': return state - 1;
    case 'RESET': return 0;
    default: return state;
  }
}
```

```
const [count, dispatch] = useReducer(reducer, 0);
dispatch({ type: 'INCREMENT' });
```

Interview tip: useReducer + Context is a lightweight alternative to Redux for medium complexity.

Q32

ADVANCED

Explain Next.js Middleware and its use cases.

Answer:

Middleware runs on the Edge before requests complete — can redirect, rewrite URLs, set headers, check auth. Runs before page rendering with near-zero latency because it executes at CDN edge nodes globally.

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(req: NextRequest) {
  const token = req.cookies.get('token');
  if (!token && req.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(
      new URL('/login', req.url)
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

Interview tip: Middleware is ideal for auth guards, A/B testing, geolocation-based routing, and i18n redirects.

Q33

ADVANCED

How do you implement code splitting in Next.js?

Answer:

Next.js automatically splits by page. Dynamic imports with next/dynamic enable component-level splitting for heavy components like charts, editors, or map libraries. Use `ssr: false` for browser-only components.

```
import dynamic from 'next/dynamic';

// Heavy chart only loaded when rendered
const Chart = dynamic(
  () => import('../components/Chart'),
  {
    loading: () => <p>Loading chart...</p>,
    ssr: false, // browser-only component
  }
);

export default function Dashboard() {
```

```

return (
  <main>
    <h1>Dashboard</h1>
    <Chart data={data} />
  </main>
);
}

```

Interview tip: Use `ssr: false` for components using `window`, `document`, or other browser-only APIs.

Q34

ADVANCED

How would you implement authentication in Next.js?

Answer:

Best practice: use Auth.js (NextAuth). For custom: store JWT in httpOnly cookies (not localStorage — XSS-safe). Use Middleware to protect routes. Server Components read cookies directly; Client Components use a session hook.

```

// Route handler – login
export async function POST(req: Request) {
  const { email, password } = await req.json();
  const user = await validateUser(email, password);
  if (!user)
    return Response.json({ error: 'Unauthorized' }, { status: 401 });

  const token = signJWT({ id: user.id, role: user.role });
  const res = Response.json({ ok: true });
  res.headers.set('Set-Cookie',
    `token=${token}; HttpOnly; Secure; SameSite=Strict; Path=/; Max-Age=3600`
  );
  return res;
}

```

Interview tip: Never store JWTs in localStorage — they are vulnerable to XSS attacks. Always use httpOnly cookies.

Q35

ADVANCED

What are Web Vitals (LCP, CLS, INP) and why are they important in Next.js?

Answer:

Core Web Vitals are Google's metrics for measuring user experience. LCP (Largest Contentful Paint) measures loading performance (aim < 2.5s). CLS (Cumulative Layout Shift) measures visual stability (aim < 0.1). INP (Interaction to Next Paint) measures interactivity/responsiveness (aim < 200ms). They heavily impact SEO.

```

import { useReportWebVitals } from 'next/web-vitals'

export function WebVitals() {
  useReportWebVitals((metric) => {
    if (metric.name === 'LCP') {
      console.log('LCP:', metric.value);
      // Send to analytics
    }
  })
}

```

Interview tip: Next.js optimizes these automatically via `next/image` (CLS/LCP), `next/font` (CLS), and Server Components (INP).

Phase 3: Node.js & Express

9 Q&As

Q36

BASIC

How does Node.js handle async if it is single-threaded?

Answer:

Node.js delegates async operations (file I/O, network, timers) to libuv's thread pool and the OS. The single-threaded event loop continuously checks for completed operations and runs callbacks. This allows handling thousands of concurrent connections without threads.

```
const fs = require('fs');

// Non-blocking – delegates to OS
fs.readFile('large.txt', 'utf8', (err, data) => {
  // Runs when I/O completes
  console.log('Done:', data.length);
});

console.log('Runs first – sync'); // runs immediately
```

Interview tip: CPU-bound tasks block the event loop — use `worker_threads` or `child_process` for those.

Q37

BASIC

What is Express middleware and how does it work?

Answer:

Middleware are functions with access to `req`, `res`, and `next()`. They form a pipeline — each can process the request and pass it forward via `next()`. Order matters. Types: application-level, router-level, error-handling (4 params), built-in, and third-party.

```
const app = express();

// Logger middleware
app.use((req, res, next) => {
  console.log(req.method + ' ' + req.path);
  next();
});

// Auth middleware
app.use('/api', requireAuth);

// Error handler (4 params – must be last)
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});
```

Interview tip: Error-handling middleware MUST have exactly 4 parameters (`err`, `req`, `res`, `next`) to work.

Q38

INTERMEDIATE

How do you structure a large Node.js application?

Answer:

Layered architecture: Routes (HTTP layer) -> Controllers (req/res handling) -> Services (business logic) -> Repositories/Models (data access). This separation makes code testable, maintainable, and scalable.

```
// routes/user.routes.ts
router.post('/users',
```



```

    validate(createUserSchema),
    userController.create
  );

// controllers/user.controller.ts
async create(req, res, next) {
  try {
    const user = await userService.create(req.body);
    res.status(201).json(user);
  } catch(e) { next(e); }
}

// services/user.service.ts
async create(data) {
  await this.verifyEmailUnique(data.email);
  return userRepository.save(data);
}

```

Interview tip: Never put business logic in controllers or routes — it kills testability.

Q39

INTERMEDIATE

Explain JWT authentication flow.

Answer:

1) User submits credentials. 2) Server validates and signs a JWT with secret. 3) Client stores token and sends it in Authorization header. 4) Server verifies token on protected routes. 5) Short-lived access token (15min); long-lived refresh token in httpOnly cookie.

```

// Sign on login
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '15m' }
);

// Verify middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ error: 'Invalid token' }); }
}

```

Interview tip: Access token: 15 min in memory/header. Refresh token: 7-30 days in httpOnly cookie.

Q40

INTERMEDIATE

What is RBAC and how do you implement it in Express?

Answer:

Role-Based Access Control restricts routes based on user roles. Each role has a set of permissions. Middleware checks the user's role before granting access. Decouples access logic from business logic.

```

const permissions = {
  admin: ['read', 'write', 'delete'],
  editor: ['read', 'write'],
  viewer: ['read'],
};

function authorize(...perms) {

```

```

return (req, res, next) => {
  const userPerms = permissions[req.user.role] || [];
  const allowed = perms.every(p => userPerms.includes(p));
  if (!allowed)
    return res.status(403).json({ error: 'Forbidden' });
  next();
};
}

router.delete('/posts/:id',
  authenticate,
  authorize('delete'),
  postController.delete
);

```

Interview tip: For complex permission systems, consider Casbin or OPA (Open Policy Agent).

Q41

INTERMEDIATE

How do you handle errors globally in Express?

Answer:

Use a centralized error handler as the last middleware. Wrap async handlers with a utility to catch promise rejections. Define custom error classes for HTTP statuses. Never send stack traces to clients in production.

```

// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Async wrapper – catches promise rejections
const catchAsync = fn =>
  (req, res, next) =>
    Promise.resolve(fn(req, res, next)).catch(next);

// Global handler
app.use(err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({
    error: err.message,
    ...(process.env.NODE_ENV !== 'production'
      && { stack: err.stack })
  });
});

```

Interview tip: Express only catches sync errors automatically — always handle async with catchAsync or try/catch.

Q42

ADVANCED

How do you implement rate limiting in a distributed Node.js system?

Answer:

express-rate-limit handles single-server limits. For distributed systems (multiple servers), use Redis as a shared store so rate limits apply across all instances consistently.

```

const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis');

const limiter = rateLimit({

```

```

windowMs: 15 * 60 * 1000, // 15 minutes
max: 100,
standardHeaders: true,
store: new RedisStore({
  client: redisClient,
  prefix: 'rl:',
}),
keyGenerator: req => req.user?.id || req.ip,
});

// Stricter limit on auth routes
const authLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 5, // 5 attempts per minute
});

app.use('/api/', limiter);
app.use('/api/auth/', authLimiter);

```

Interview tip: Apply stricter limits on /login and /register to prevent brute-force attacks.

Q43

ADVANCED

Explain Node.js streams and when to use them.

Answer:

Streams process data in chunks rather than loading everything into memory. Types: Readable, Writable, Duplex, Transform. Critical for large file handling, video streaming, and data pipelines — prevents memory overflow.

```

const fs = require('fs');
const zlib = require('zlib');

// Stream large file through gzip compression
// Without streams: entire file in RAM
// With streams: chunk by chunk
fs.createReadStream('large.csv')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('large.csv.gz'))
  .on('finish', () => console.log('Compressed'));

// Stream DB query results to HTTP response
app.get('/export', (req, res) => {
  res.setHeader('Content-Type', 'text/csv');
  db.queryStream('SELECT * FROM orders')
    .pipe(csvTransform)
    .pipe(res);
});

```

Interview tip: Never use fs.readFile for large files — always stream them.

Q44

ADVANCED

How do you implement WebSockets for real-time features?

Answer:

WebSockets provide full-duplex persistent connections — ideal for chat, live updates, collaborative tools. HTTP polling wastes bandwidth; WebSockets push instantly. Use Socket.IO for production. Scale across multiple servers with Redis pub/sub adapter.

```

const { Server } = require('socket.io');
const io = new Server(httpServer);

```

```
io.on('connection', socket => {
  socket.on('join-room', roomId => {
    socket.join(roomId);
    socket.to(roomId).emit('user-joined', socket.id);
  });

  socket.on('message', ({ roomId, text }) => {
    io.to(roomId).emit('message', {
      from: socket.id,
      text,
      at: new Date().toISOString()
    });
  });

  socket.on('disconnect', () => {
    console.log('Disconnected:', socket.id);
  });
});
```

Interview tip: Use [@socket.io/redis-adapter](#) to sync events across multiple Node.js instances.

Phase 4: MongoDB & PostgreSQL

8 Q&As

Q45

BASIC

When should you use MongoDB vs PostgreSQL?

Answer:

MongoDB excels with flexible documents, nested/hierarchical data, rapidly evolving schemas, and horizontal scale. PostgreSQL excels with relational data, complex joins, ACID transactions, and strong consistency requirements. Many production apps use both.

```
// MongoDB – flexible nested document
{
  _id: '123',
  name: 'Nazmul',
  addresses: [
    { type: 'home', city: 'Dhaka' },
    { type: 'work', city: 'Mirpur' }
  ]
}

// PostgreSQL – normalized
-- users: id, name
-- addresses: id, user_id FK, type, city
```

Interview tip: Need ACID transactions and complex joins? PostgreSQL. Need schema flexibility and scale? MongoDB.

Q46

BASIC

What are database indexes and why do they matter?

Answer:

Indexes are data structures (typically B-trees) that allow the database to find rows without full table scans. Without an index on a WHERE column: $O(n)$ scan. With an index: $O(\log n)$. Trade-off: indexes speed reads but slow writes and use storage.

```
-- PostgreSQL
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_posts_user_created
  ON posts(user_id, created_at DESC); -- composite

-- MongoDB
db.users.createIndex({ email: 1 }, { unique: true });
db.orders.createIndex(
  { userId: 1, createdAt: -1 }
);
```

Interview tip: Use EXPLAIN ANALYZE (Postgres) or .explain('executionStats') (MongoDB) to verify index usage.

Q47

INTERMEDIATE

Explain MongoDB's aggregation pipeline.

Answer:

The aggregation pipeline processes documents through sequential stages, each transforming output. Key stages: \$match (filter early), \$group (aggregate), \$project (reshape), \$sort, \$limit, \$lookup (join), \$unwind (flatten arrays).

```
db.orders.aggregate([
  { $match: { status: 'completed' } },
  { $group: {
    _id: '$userId',
    totalSpent: { $sum: '$amount' },
    orderCount: { $sum: 1 }
  }},
  { $lookup: {
    from: 'users',
    localField: '_id',
    foreignField: '_id',
    as: 'user'
  }},
  { $unwind: '$user' },
  { $sort: { totalSpent: -1 } },
  { $limit: 10 }
]);
```

Interview tip: Always put \$match as early as possible in the pipeline to reduce document count in later stages.

Q48

INTERMEDIATE

What are PostgreSQL transactions and ACID properties?

Answer:

ACID: Atomicity (all or nothing), Consistency (always valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). Transactions group operations so partial failures roll back entirely.

```
BEGIN;

UPDATE accounts
  SET balance = balance - 500
  WHERE id = 1;

UPDATE accounts
  SET balance = balance + 500
  WHERE id = 2;

COMMIT; -- Both succeed or both roll back

-- Savepoints for partial rollback
SAVEPOINT before_step;
-- ... more operations ...
ROLLBACK TO SAVEPOINT before_step;
```

Interview tip: MongoDB 4.0+ supports multi-document ACID transactions, but they add overhead vs SQL.

Q49

INTERMEDIATE

What is the N+1 query problem and how do you solve it?

Answer:

N+1 occurs when you fetch N records then make 1 additional query per record. Example: 100 posts + 1 query per author = 101 queries. Fix: SQL JOINS / ORM eager loading, DataLoader batching, or MongoDB \$lookup.

```
// N+1 problem
const posts = await Post.findAll(); // 1 query
for (const post of posts) {
  // N queries!
  const author = await User.findById(post.userId);
}
```

```
// Fix: eager load (1 JOIN query)
const posts = await Post.findAll({
  include: [{ model: User, as: 'author' }]
});

// MongoDB fix: $lookup
db.posts.aggregate([
  { $lookup: {
    from: 'users', localField: 'userId',
    foreignField: '_id', as: 'author'
  }}
]);
```

Interview tip: Enable query logging in development to detect N+1 — Mongoose debug mode, Sequelize logging: true.

Q50

ADVANCED

Explain read replicas and database sharding.

Answer:

Read Replicas: copies of primary DB for read traffic. Writes to primary, reads from replicas. Reduces read pressure. Sharding: partitions data across multiple DBs by shard key (range or hash). Each shard is independent. Enables horizontal scaling beyond single-machine limits.

```
// Read/write separation
await primaryPool.query('INSERT INTO orders...');
await replicaPool.query('SELECT * FROM orders WHERE userId = $1', [id]);

// MongoDB sharding
sh.enableSharding('mydb');
sh.shardCollection('mydb.users', { region: 'hashed' });
// Asia users -> Shard A
// Europe users -> Shard B
// Queries route automatically
```

Interview tip: Choose shard keys carefully — a bad key causes hot spots where one shard gets all the traffic.

Q51

ADVANCED

How do you optimize a slow SQL query?

Answer:

Steps: 1) EXPLAIN ANALYZE — see the query plan. 2) Check for missing indexes. 3) Avoid SELECT * — only fetch needed columns. 4) Avoid functions on indexed columns in WHERE. 5) Paginate large results. 6) Use covering indexes. 7) Consider query result caching with Redis.

```
-- Slow: function prevents index use
SELECT * FROM users
WHERE LOWER(email) = 'test@example.com';

-- Fix: index on expression
CREATE INDEX idx_lower_email
ON users(LOWER(email));

-- Better projection
SELECT id, name, email FROM users
WHERE LOWER(email) = 'test@example.com';

-- Check plan
EXPLAIN ANALYZE
SELECT id, name FROM users
WHERE email = 'x@y.com';
```

Q52

ADVANCED

Explain MongoDB schema design: embedding vs referencing.

Answer:

Embed when: data always accessed together, bounded one-to-many, child data rarely changes alone.
Reference when: many-to-many, data accessed independently, unbounded arrays, or documents risk exceeding 16MB.

```
// Embed — post with few comments
{
  _id: 'post1',
  title: 'Hello World',
  comments: [
    { author: 'Ali', text: 'Great!' }
  ]
}

// Reference — user with millions of orders
// users: { _id, name, email }
// orders: { _id, userId: ObjectId, amount }

// Query with $lookup
db.orders.aggregate([
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'user'
  }}
]);
```

Interview tip: MongoDB documents have a hard 16MB limit — never embed unbounded arrays.

Q53

BASIC

What is Big O notation? Give examples.

Answer:

Big O describes time/space complexity as input size grows. $O(1)$ constant, $O(\log n)$ binary search, $O(n)$ linear, $O(n \log n)$ merge sort, $O(n^2)$ nested loops, $O(2^n)$ exponential.

```
// O(1) - hash lookup
const map = new Map();
map.get('key'); // constant

// O(n) - linear scan
array.find(x => x === target);

// O(log n) - binary search
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid] === target) return mid;
    arr[mid] < target ? lo = mid+1 : hi = mid-1;
  }
  return -1;
}
```

Interview tip: In interviews: state time complexity, then space complexity — both matter.

Q54

INTERMEDIATE

Two Sum problem — optimal approach.

Answer:

Naive: $O(n^2)$ nested loops. Optimal: one pass with a hash map — $O(n)$ time, $O(n)$ space. For each number, check if its complement (target - num) already exists in the map. This pattern solves dozens of array problems.

```
function twoSum(nums, target) {
  const map = new Map(); // value -> index

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];

    if (map.has(complement)) {
      return [map.get(complement), i];
    }
    map.set(nums[i], i);
  }
  return [];
}

// twoSum([2,7,11,15], 9) -> [0,1]
// O(n) time, O(n) space
```

Interview tip: The complement pattern in a hash map is the most common interview optimization technique.

Design a URL shortener system.**Answer:**

Requirements: shorten URLs, redirect fast, handle 1M+ requests/day. Components: API server, ID generator (base62 of auto-increment), Redis (fast lookup), PostgreSQL (persistent store), CDN (redirect at edge).

```
// ID generation (base62)
const CHARS = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ';
function encode(num) {
  let result = '';
  while (num > 0) {
    result = CHARS[num % 62] + result;
    num = Math.floor(num / 62);
  }
  return result.padStart(7, '0');
}

// Request flow:
// POST /shorten -> generate ID -> store in DB + Redis
// GET /:id -> Redis lookup (cache hit ~99%)
// -> 301 redirect to original URL
```

Interview tip: Always start system design: requirements -> estimation -> high-level -> deep dive.

What is the difference between vertical and horizontal scaling?**Answer:**

Vertical: add more CPU/RAM to one server. Simple but has hardware ceiling and single point of failure. Horizontal: add more servers behind a load balancer. No single point of failure, virtually unlimited scale, but requires stateless application design.

```
// Horizontal scaling requires stateless services
// Bad: session in memory (only one server has it)
app.use(session({ secret: 'x', resave: false }));

// Good: session in Redis (shared across servers)
app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET,
  resave: false,
}));

// JWT is inherently stateless – even better
```

Interview tip: Horizontal scaling + stateless design is the foundation of cloud-native architecture.

Explain caching strategies: cache-aside, write-through, write-behind.**Answer:**

Cache-aside (lazy loading): check cache first; on miss, load DB and populate cache. Most common. Write-through: write to both cache and DB simultaneously — consistent but slower writes. Write-behind: write to cache immediately, async persist to DB — fastest writes, risk of loss on crash.

```
// Cache-aside pattern
async function getUser(id) {
```

```

const key = 'user:' + id;
const cached = await redis.get(key);
if (cached) return JSON.parse(cached);

const user = await db.findById(id);
await redis.setex(key, 3600, JSON.stringify(user));
return user;
}

// Invalidate on update
async function updateUser(id, data) {
  await db.update(id, data);
  await redis.del('user:' + id);
}

```

Interview tip: Always set TTL on cached entries — never cache without expiry.

Q58

ADVANCED

How would you design a real-time notification system?

Answer:

Components: event producers, message broker (Kafka for scale), notification service (processes + routes), delivery channels (WebSocket for online, push/email/SMS for offline). Use fan-out pattern for broadcasting to many users.

```

// Producer emits event
await kafka.send({
  topic: 'notifications',
  messages: [{ value: JSON.stringify({
    type: 'ORDER_PLACED',
    userId: user.id,
    orderId: order.id
  })}]
});

// Consumer routes notifications
kafka.on('message', async msg => {
  const event = JSON.parse(msg.value);
  // WebSocket for online users
  io.to('user:' + event.userId).emit('notification', {
    type: event.type, message: 'Order confirmed!'
  });
  // Email for offline users
  if (!onlineUsers.has(event.userId))
    await emailQueue.add(event);
});

```

Interview tip: Separate notification routing from delivery — use dedicated workers per channel.

Q59

ADVANCED

Explain the CAP theorem.

Answer:

CAP states a distributed system can guarantee only 2 of 3: Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (works despite network failures). Since partitions are inevitable in distributed systems, real choice is CP or AP.

```

// CP systems: MongoDB, ZooKeeper, HBase
// During partition: refuse writes to stay consistent
// Use when: bank balances, inventory counts

```

```
// AP systems: Cassandra, DynamoDB, CouchDB
// During partition: serve potentially stale data
// Use when: social feeds, product catalog

// Tunable consistency example (Cassandra):
CONSISTENCY QUORUM; -- majority of nodes must agree
CONSISTENCY EVENTUAL; -- fast, may read stale
```

Interview tip: Modern systems often have configurable consistency (e.g., MongoDB readConcern/writeConcern).

Q60

ADVANCED

Explain database connection pooling.

Answer:

Creating a new DB connection is expensive (TCP handshake, auth, memory). Connection pooling maintains a pool of pre-established connections reused across requests. Without pooling, high traffic exhausts DB connections. pg-pool (Postgres), Mongoose (MongoDB) pool by default.

```
// PostgreSQL with pg pool
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  max: 20, // max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

// Reuses existing connection from pool
const result = await pool.query(
  'SELECT * FROM users WHERE id = $1', [id]
);
```

*Interview tip: Tune max pool size based on DB max_connections. Rule: pool size = (number of cores * 2) + effective spindle count.*

Phase 6: Scalable Architecture & Production DevOps

12 Q&As

Q61

BASIC

What is the difference between monolithic and microservices architecture?

Answer:

Monolith: single deployable unit, all features in one codebase — simple for small teams. Microservices: independently deployable services, each owning its domain and database — enables independent scaling and team autonomy but adds operational complexity.

```
// Monolith – one deployment
app/
  src/
    auth/
    users/
    products/
    payments/

// Microservices – many deployments
auth-service/    -> :3001
user-service/    -> :3002
product-service/ -> :3003
payment-service/ -> :3004
// Each communicates via HTTP/gRPC/events
// Each has its own database
```

Interview tip: Most startups: start monolith, extract microservices when specific pain points appear.

Q62

BASIC

What is Docker and why is it used?

Answer:

Docker packages an application and all its dependencies into a container — a portable unit that runs identically everywhere. Solves 'works on my machine' problems. Containers share the host OS kernel (unlike VMs) so they start in seconds and use less resources.

```
# Multi-stage Dockerfile for Next.js
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM deps AS builder
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=deps /app/node_modules ./node_modules
EXPOSE 3000
CMD ["npm", "start"]
```

Interview tip: Multi-stage builds keep images small by excluding build tools from the final image.

What is CI/CD and why does it matter?

Answer:

CI (Continuous Integration): automatically test and build on every push, catching bugs early. CD (Continuous Deployment): automatically deploy passing builds to production. Together they reduce manual errors, speed up releases, and ensure consistent deployments.

```
# .github/workflows/deploy.yml
name: CI/CD Pipeline
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm test
      - run: npm run build
      - name: Deploy
        run: |
          ssh user@server 'cd /app &&
            git pull && npm ci &&
            npm run build &&
            pm2 restart all'
```

Interview tip: Never deploy to production without automated tests passing in CI.

What is load balancing? Explain load balancing algorithms.

Answer:

Load balancing distributes requests across multiple servers to prevent overload and enable scaling. Algorithms: Round Robin (rotate equally), Least Connections (route to least busy), IP Hash (consistent routing per user for session stickiness), Weighted (more traffic to stronger servers).

```
# Nginx load balancer config
upstream backend {
    least_conn;

    server app1.example.com:3000 weight=3;
    server app2.example.com:3000 weight=2;
    server app3.example.com:3000 weight=1;

    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }
}
```

Q65

INTERMEDIATE

How does Redis work as a cache? Explain eviction policies.

Answer:

Redis is an in-memory key-value store with sub-millisecond latency. When memory is full, eviction policies determine what to remove. LRU (Least Recently Used) is most common for caches. `allkeys-lru` evicts any key; `volatile-lru` only evicts keys with TTL set.

```
const redis = require('ioredis');
const client = new redis(process.env.REDIS_URL);

// Set with 1-hour TTL
await client.setex('product:123', 3600,
  JSON.stringify(product));

// Get
const raw = await client.get('product:123');
if (raw) return JSON.parse(raw);

// Atomic operations
await client.incr('page:views:home');
await client.lpush('recent:products', productId);
await client.ltrim('recent:products', 0, 9); // keep 10
```

Interview tip: Set `maxmemory` and `maxmemory-policy` in `redis.conf`. `allkeys-lru` is the standard cache policy.

Q66

INTERMEDIATE

What are message queues and when should you use them?

Answer:

Message queues decouple producers from consumers and handle async workloads. Use when: tasks are time-consuming (email, video processing), you need retry logic on failure, you need to absorb traffic spikes, or services need to communicate without tight coupling.

```
const { Queue, Worker } = require('bullmq');

// Producer – add to queue
const emailQueue = new Queue('email', { connection: redis });

await emailQueue.add('welcome', { userId }, {
  attempts: 3,
  backoff: { type: 'exponential', delay: 1000 }
});

// Consumer – process jobs
new Worker('email', async job => {
  const { userId } = job.data;
  const user = await db.findById(userId);
  await mailer.sendWelcome(user.email);
}, { connection: redis });
```

Interview tip: BullMQ (Redis-based) is great for Node.js. For massive event streaming, use Apache Kafka.

Explain the production infrastructure flow for a scalable app.

Answer:

Request journey: User -> CDN (static + edge cache) -> Load Balancer -> App Servers (stateless) -> Cache Layer (Redis) -> Database (primary writes / replica reads). Background jobs go through message queues to workers.

```
// Production flow:
// 1. User hits CDN (Cloudflare/CloudFront)
//    -> cached: serve immediately
//    -> miss: forward to origin

// 2. Load Balancer (Nginx/AWS ALB)
//    -> distributes to app server pool

// 3. App Server (stateless Next.js/Node)
//    -> check Redis cache
//    -> cache hit: return immediately
//    -> cache miss: query PostgreSQL replica
//    -> write operations: PostgreSQL primary

// 4. Background tasks:
//    -> BullMQ/Kafka -> Workers
//    -> (email, notifications, analytics)
```

Interview tip: Always make app servers stateless — store sessions, cache, and feature flags in Redis.

What is Kubernetes and how does it orchestrate containers?

Answer:

Kubernetes (K8s) manages containers at scale: scheduling across nodes, self-healing (restarts failed pods), rolling deployments (zero downtime), HPA (auto-scaling), load balancing, service discovery, and secret management.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: { app: api }
  template:
    spec:
      containers:
        - name: api
          image: myregistry/api:v1.2.3
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 500m, memory: 512Mi }
---
# Auto-scale 3-20 pods based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 3
  maxReplicas: 20
  metrics:
```



```
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
```

Interview tip: Start with Docker Compose for local dev. Graduate to Kubernetes when managing 5+ services.

Q69

ADVANCED

How would you design a system for 1 million concurrent users?

Answer:

Layer the architecture: CDN -> Load Balancer -> Stateless App Servers (auto-scaling) -> Redis Cache Cluster -> DB (primary + read replicas + sharding at extreme scale) -> Message Queues -> Background Workers -> Monitoring.

```
// Capacity estimation for 1M users:
// Peak: ~50K req/sec (assume 5% concurrent)
// Avg response: 100ms -> 5000 RPS per server
// -> Need ~10 app servers at peak
// DB reads: 80% -> route to 3 replicas
// DB writes: 20% -> primary
// Cache hit rate target: 95%+

// Auto-scaling rule:
// Scale out when CPU > 70% for 2 minutes
// Scale in when CPU < 30% for 5 minutes
// Min: 3 instances (HA)
// Max: 50 instances (cost cap)
```

Interview tip: Always estimate: traffic -> server count -> DB connections -> cache hit rate -> infrastructure cost.

Q70

ADVANCED

Explain distributed tracing and observability.

Answer:

Observability has three pillars: Metrics (Prometheus), Logs (centralized logging), Traces (distributed request tracing). Distributed tracing tracks a request across services using a shared trace ID propagated in headers. Use OpenTelemetry as the standard instrumentation layer.

```
// OpenTelemetry setup
const { trace } = require('@opentelemetry/api');

async function processOrder(orderId) {
  const tracer = trace.getTracer('order-service');
  const span = tracer.startSpan('processOrder');
  span.setAttribute('order.id', orderId);

  try {
    const order = await db.findOrder(orderId);
    span.addEvent('order.fetched');

    await paymentService.charge(order);
    span.setStatus({ code: SpanStatusCode.OK });
  } catch (err) {
    span.recordException(err);
    span.setStatus({ code: SpanStatusCode.ERROR });
    throw err;
  } finally {
    span.end();
  }
}
```

```
}  
}
```

Interview tip: Key SLIs to monitor: latency (p50/p95/p99), error rate (%), throughput (RPS), saturation (CPU/mem).

Q71

ADVANCED

What is the Saga pattern in microservices?

Answer:

Saga manages distributed transactions across microservices that each own their own database (no shared DB, no SQL transactions). Choreography: services emit events that trigger other services. Orchestration: a central orchestrator calls services and handles compensating transactions on failure.

```
// Orchestration Saga  
class OrderSaga {  
  async execute(orderId) {  
    try {  
      await inventoryService.reserve(orderId);  
      await paymentService.charge(orderId);  
      await shippingService.schedule(orderId);  
      await notifyService.send(orderId);  
    } catch (err) {  
      // Compensate – rollback in reverse  
      await shippingService.cancel(orderId);  
      await paymentService.refund(orderId);  
      await inventoryService.release(orderId);  
      throw err;  
    }  
  }  
}
```

Interview tip: Compensating transactions are the microservices equivalent of SQL ROLLBACK.

Q72

ADVANCED

What are common production engineering mistakes to avoid?

Answer:

1) No monitoring — blind when things break. 2) Hardcoded secrets — use environment variables and secret managers. 3) No rate limiting — open to brute force. 4) Unhandled promise rejections — crash Node processes. 5) No backups — catastrophic data loss. 6) No CI/CD — error-prone manual deploys. 7) Premature microservices — adds complexity before it's needed.

```
// Handle all unhandled rejections  
process.on('unhandledRejection', (reason, promise) => {  
  logger.error('Unhandled rejection:', reason);  
  // Graceful shutdown  
  server.close(() => process.exit(1));  
});  
  
// Validate env vars on startup  
const required = ['DB_URL', 'JWT_SECRET', 'REDIS_URL'];  
for (const key of required) {  
  if (!process.env[key])  
    throw new Error(key + ' env var not set');  
}  
  
// Graceful shutdown  
process.on('SIGTERM', async () => {  
  await server.close();  
  await db.end();  
  process.exit(0);  
});
```

```
});
```

Interview tip: Add a production readiness checklist: monitoring, logging, backups, secrets, rate limits, graceful shutdown.

Phase 7: Testing & Quality Assurance

3 Q&As

Q73

BASIC

What is the difference between Unit, Integration, and End-to-End (E2E) testing?

Answer:

Unit: tests a single function/component in isolation (Jest). Integration: tests how multiple units work together (React Testing Library, API tests). E2E: simulates a real user journey in a real browser (Cypress, Playwright).

```
// Unit
expect(sum(1, 2)).toBe(3);

// Integration
render(<LoginForm />);
userEvent.type(screen.getByLabelText('Email'), 'test@test.com');

// E2E
await page.goto('/login');
await page.fill('#email', 'test@test.com');
```

Interview tip: The Testing Pyramid suggests many Unit tests, some Integration tests, and fewer E2E tests (as they are slow and brittle).

Q74

INTERMEDIATE

Explain React Testing Library vs Enzyme.

Answer:

Enzyme tests implementation details (state, props, component internals). React Testing Library tests behavior from the user's perspective (finding elements by role, text, or label). RTL is the modern standard because it makes tests resilient to refactoring.

```
// Bad (Enzyme approach - testing internals)
wrapper.setState({ count: 1 });
expect(wrapper.state('count')).toBe(1);

// Good (RTL approach - testing behavior)
render(<Counter />);
fireEvent.click(screen.getByRole('button', { name: /increment/i }));
expect(screen.getByText('Count: 1')).toBeInTheDocument();
```

Interview tip: Always try to query by accessibility roles first (e.g. getByRole). It ensures your app is accessible too.

Q75

ADVANCED

How do you mock API calls in tests?

Answer:

Avoid making real network requests in tests. You can mock fetch/axios directly with Jest (jest.mock). Better yet, use MSW (Mock Service Worker) which intercepts network requests at the network level and returns mock responses — it works identically for tests and local development.

```
import { rest } from 'msw';
import { setupServer } from 'msw/node';

const server = setupServer(
  rest.get('/api/user', (req, res, ctx) => {
```

```
    return res(ctx.json({ name: 'John' }));  
  })  
);  
  
beforeAll(() => server.listen());  
afterEach(() => server.resetHandlers());  
afterAll(() => server.close());
```

Interview tip: MSW is the industry standard for mocking APIs in modern frontend testing. Mention it to stand out.
